个人答辩报告

12312420 王振宇

SUSTC

May 13, 2025

Content

1. 负责工作简述
2. OutputModule
3. ClockDivider
4. SegGenerator
5. SegDisplay
6. ALU

负责工作简述

1. OutputModule: CPU 输出模块，从寄存器中读取数据
   - ClockDivider: 时钟分频器
   - SegGenerator: 生成位选信号
   - SegDisplay: 生成段选信号，输出内容控制（2/10/16 进制）
2. ALU: 算数逻辑单元

OutputModule

```
module OutputModule(
    input clk,
    input rst,
    input en_output,
    input [31:0] reg_a7,
    input [31:0] output_data,
    output [7:0] seg1,
    output [7:0] seg2,
    output [7:0] led,
    output wire [7:0] an   //control the 8-segment
);

parameter PERIOD_SEG = 1<<16;

wire [31:0] val,tmp_reg_a7;
wire [3:0] select;
wire clk_dis;
```

```
assign val = (en_output ? output_data: val);
assign tmp_reg_a7 = (en_output ? reg_a7: tmp_reg_a7);

SegGenerator uut_seg_gen (
    ...
);

ClockDivider uut_clk_display (// 2^16 bits for display
    ...
);

SegDisplay uut_seg_display (// display module
    ...
);
endmodule
```

Figure: Code of OutputModule

ClockDivider

```
module ClockDivider(
    input clk,
    input rst,
    input [31:0] period,
    output reg clk_out
);

reg [24:0] cnt;
```

```
always @(posedge clk or negedge rst)
        begin
    if (~rst) begin
        cnt <= 0;
        clk_out <= 0;
    end else begin
        if (cnt == (period>>1)-1)
                begin
            cnt <= 0;
            clk_out <= ~clk_out;
        end else begin
            cnt <= cnt + 1;
        end
    end
end
endmodule
```

Figure: Code of ClockDivider

ClockDivider

```
module tb_ClockDivider;

reg          clk;
reg          rst;
reg [31:0]   period;

wire         clk_out;

// Instantiate DUT
ClockDivider uut (
    .clk      (clk),
    .rst      (rst),
    .period   (period),
    .clk_out  (clk_out)
);

initial clk = 0;
always #5 clk = ~clk;
```

```
initial begin
    $dumpfile("tb_ClockDivider.vcd");
    $dumpvars(0, tb_ClockDivider);

    rst    = 0;
    period = 32'd20;   // output clk
          toggles every (20/2)=10
          cycles -> 100 ns period
    #10;
    rst    = 1;
    #500;
    period = 32'd10;   // output clk
          toggles every (10/2)=5
          cycles -> 50 ns period
    #500;

    $finish;
end

endmodule
```

Figure: Testbench of ClockDivider

ClockDivider

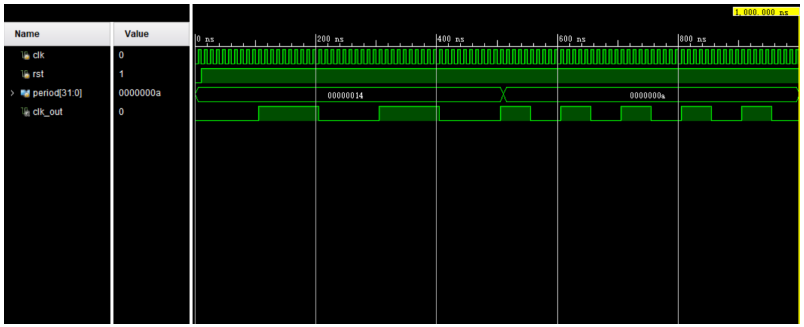

Figure: Waveform of ClockDivider

Clock Divider 波形解析

- 复位阶段：`rst=0` 时 `clk_out` 保持低；`rst=1` 后分频器开始工作
- 第一段（`period=20`）：输出翻转间隔 10 个输入沿 $\rightarrow$ 半周期 $10 \times 5\,\text{ns} = 50\,\text{ns}$ $\rightarrow$ 完整周期 $100\,\text{ns}$
- 第二段（`period=10`）：输出翻转间隔 5 个输入沿 $\rightarrow$ 半周期 $5 \times 5\,\text{ns} = 25\,\text{ns}$ $\rightarrow$ 完整周期 $50\,\text{ns}$

SegGenerator

```
module SegGenerator(
    input rst,
    input clk_dis,
    output reg [3:0] select,
    output wire [7:0] an
);
always @(posedge clk_dis or negedge
      rst) begin
    if (~rst) begin
        select <= 4'd0;
    end else begin
        if (select == 4'd7) begin
            select <= 4'd0;
        end else begin
            select <= select + 1;
        end
    end
end
```

```
assign an = (~rst) ? 8'b00000000 :
    (select == 4'd0) ? 8'b10000000 :
    (select == 4'd1) ? 8'b01000000 :
    (select == 4'd2) ? 8'b00100000 :
    (select == 4'd3) ? 8'b00010000 :
    (select == 4'd4) ? 8'b00001000 :
    (select == 4'd5) ? 8'b00000100 :
    (select == 4'd6) ? 8'b00000010 :
    (select == 4'd7) ? 8'b00000001 :
            8'b11111111;
endmodule
```

Figure: Code of SegGenerator

SegGenerator

```
// 1. Declarations
module tb_SegGenerator;
  // inputs
  reg        rst;
  reg        clk_dis;
  // outputs
  wire [3:0] select;
  wire [7:0] an;
// 2. DUT instantiation
SegGenerator uut (
  .rst     (rst),
  .clk_dis (clk_dis),
  .select  (select),
  .an      (an)
);
// 50 MHz clk_dis → toggle every 10
      ns
initial clk_dis = 0;
always #10 clk_dis = ~clk_dis;
```

```
// 3. Reset behavior
initial begin
  rst = 0;  #20;
  rst = 1;  #1;
  if (select!==4'd0||an!==8'
        b10000000) $fatal("...");
// 4. Sequence through 10 ticks
  integer i;
  reg [3:0] expected_sel = 4'd0;
  reg [7:0] expected_an  = 8'
        b10000000;
  for (i=1;i<=10;i=i+1) begin
    // wait one display-clock tick
    // compute next expected and
          check DUT outputs
  end
  $display("...");
  #10; $finish;
end
endmodule
```

Figure: Testbench of SegGenerator

SegGenerator

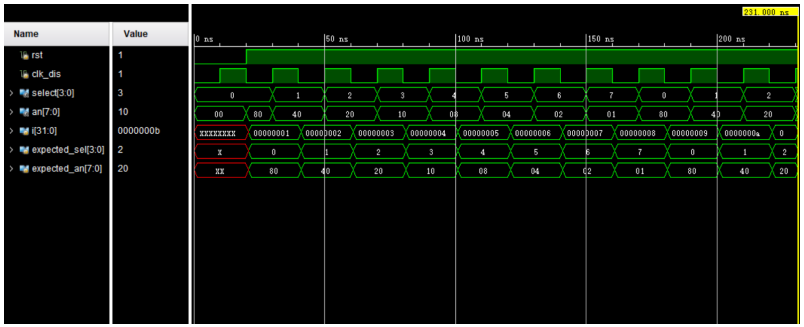

Figure: Waveform of SegGenerator

SegGenerator 波形验证

- 复位后：select=0, an=8'b10000000，与 expected 相同
- 周期 1-8：每个 clk_dis 上升沿，select 从 0 递增到 7，an 依次右移，实际值与 expected_sel/expected_an 一致
- 周期 9-10：select 回到 0/1，an 回到 8'b10000000/ 8'b01000000，匹配预期

SegDisplay

```
// 1. Header & ports
module SegDisplay(
    input  [31:0] reg_a7,
    input  [31:0] val,
    input  [3:0]  select,
    output reg [7:0] led,
    output reg [7:0] seg1,
    output reg [7:0] seg2
);
// 2. Lookup table
reg [7:0] seg_map [0:15];
initial begin
  // map hex digit → 7-segment
       pattern
  seg_map[0]=8'b11111100;
  ...
  seg_map[15]=8'b10001110;
  // (all 16 entries initialized)
end
```

```
// 3. Display logic
always @(*) begin
  case (reg_a7)
    32'd1: begin
      led = 8'b00000000;
      case (select)
        4'd0: seg1 = seg_map[(val
              /10000000) % 10];
        4'd1: seg1 = seg_map[(val
              /1000000) % 10];
        // ... digits 2-3 ...
        4'd4: seg2 = seg_map[(val
              /1000) % 10];
        // ... digits 5-7 ...
        default: begin
          seg1 = 8'b00000000;
          seg2 = 8'b00000000;
          led  = 8'b00000000;
        end
      endcase
    end
```

Figure: Code of SegDisplay

SegDisplay (cont'd)

```
    32'd1: begin
      led = 8'b00000000;
      case (select)
        4'd0: seg1 = seg_map[val
              [31:28]];
        4'd1: seg1 = seg_map[val
              [27:24]];
        // ... digits 2-3 ...
        4'd4: seg2 = seg_map[val
              [15:12]];
        // ... digits 5-7 ...
        default: begin
          seg1 = 8'b00000000;
          seg2 = 8'b00000000;
          led  = 8'b00000000;
        end
      endcase
    end
```

```
    32'd35: begin
      // raw LED = low byte of val
      seg1 = 8'b00000000;
      seg2 = 8'b00000000;
      led  = val[7:0];
    end
    default: begin
      // blank display
      seg1 = 8'b00000000;
      seg2 = 8'b00000000;
      led  = 8'b00000000;
    end
  endcase
end

endmodule
```

Figure: Code of SegDisplay (cont'd)

SegDisplay

```
module tb_SegDisplay;

// inputs
reg  [31:0] reg_a7;
reg  [31:0] val;
reg  [3:0]  select;

// outputs
wire [7:0] led;
wire [7:0] seg1;
wire [7:0] seg2;

// instantiate DUT
SegDisplay uut (
    .reg_a7 (reg_a7),
    .val    (val),
    .select (select),
    .led    (led),
    .seg1   (seg1),
    .seg2   (seg2)
);
```

```
initial begin
$dumpfile("tb_SegDisplay.vcd");
$dumpvars(0, tb_SegDisplay);
// ----- Test decimal display -----
reg_a7 = 32'd1;
val    = 32'd12345678;
for (select = 0; select < 8; select
      = select + 1) begin
  #1;
  $display("DEC sel=%0d: seg1=%b
        seg2=%b led=%b", select, seg1
        , seg2, led);
end
// ----- Test hex display -----
    ...
// ----- Test LED output -----
    ...

#1 $finish;
end
endmodule
```

Figure: Testbench of SegDisplay

SegDisplay

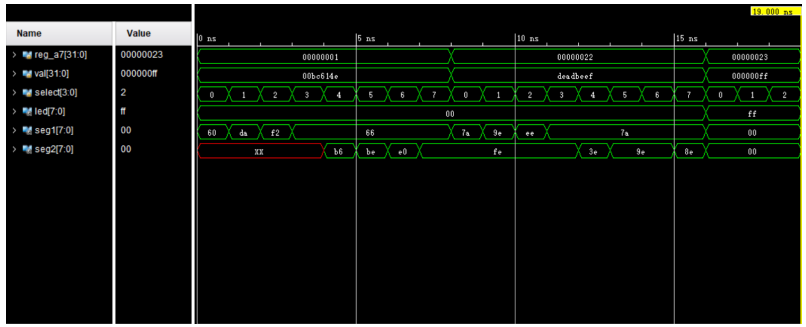

Figure: Waveform of SegDisplay

SegDisplay 波形验证

- 模式 1（reg_a7=1）：select 0-3 时 seg1 映射十进制高位，4–7 时 seg2 映射十进制低位；led=8'b00000000
- 模式 34（reg_a7=34）：select 0-7 依次选择 val 的 8 个半字节，seg1/seg2 分别输出当前半字节的八段码；led=8'b00000000
- 模式 35（reg_a7=35）：led 直接输出 val[7:0]，seg1 与 seg2 均为 8'b00000000

ALU

```
// 1. Declarations
module ALU(
    input  [31:0] read_data1,
    input  [31:0] read_data2,
    input  [31:0] imm32,
    input         alu_src,
    input  [1:0]  alu_op,
    input  [2:0]  funct3,
    input  [6:0]  funct7,
    output reg [31:0] alu_result,
    output reg    zero,
    input         is_lui,
    input         is_auipc,
    input [31:0]  pc
);

wire [31:0] operand2;
wire [4:0] shamt;  // shift amount (
    only low 5 bits valid for 32-
    bit)
```

```
// 2. Operand select
wire [31:0] operand2 = alu_src ?
      imm32 : read_data2;
wire [4:0]  shamt    = operand2
      [4:0];  // shift amount

always @(*) begin
  case (alu_op)
    2'b00: begin
      // Load/Store:
      //   alu_result = read_data1 +
            operand2
    end
```

Figure: Code of ALU

ALU (cont'd)

```
// 3. Branch (2'b01)
    2'b01: begin
      // subtract and set zero flag:
      // BEQ/BNE: eq vs neq
      // BLT/BGE: signed compare
      // BLTU/BGEU: unsigned compare
    end

// R-type (2'b10)
    2'b10: begin
      // use funct3/funct7 to select:
      // ADD/SUB, XOR, OR, AND,
      // SLL, SRL, SRA, SLT, SLTU
    end
```

```
// 4. I-type & U-type (2'b11)
    2'b11: begin
      // if is_lui: alu_result =
           operand2
      // else if is_auipc:
           alu_result = pc +
           operand2
      // else: perform ADDI, XORI,
           ORI, ANDI,
      // SLLI, SRLI, SRAI, SLTI,
           SLTIU
    end

    default: alu_result = 32'd0; //
         safe default
  endcase
end

endmodule
```

Figure: Code of ALU (cont'd)

ALU

```
// 1. Declarations
module tb_ALU;
  // inputs
  reg [31:0] read_data1, read_data2,
       imm32, pc;
  reg alu_src, is_lui, is_auipc;
  reg [1:0]  alu_op;
  reg [2:0]  funct3;
  reg [6:0]  funct7;
  // outputs
  wire [31:0] alu_result;
  wire        zero;
```

```
// 2. DUT instantiation
ALU uut (
  .read_data1(read_data1),
  .read_data2(read_data2),
  .imm32     (imm32),
  .alu_src   (alu_src),
  .alu_op    (alu_op),
  // ... other port connections ...
  .alu_result(alu_result),
  .zero      (zero)
);
```

Figure: Testbench of ALU

ALU (cont'd)

```
// 3. Tests 1-3
initial begin
  // waveform dump setup
  $dumpfile("tb_ALU.vcd");
  $dumpvars(0, tb_ALU);

  // Test 1: ADD (load/store)
  //   alu_src=0; alu_op=00;
       read_data1=15; read_data2=27;
  //   expect alu_result==42

  // Test 2: BEQ zero detection
  //   alu_op=01; funct3=000; data1
       ==data2;
  //   expect zero==1

  // Test 3: R-type SLL
  //   alu_op=10; funct3=001; shift
       left by shamt
```

```
// 4. Tests 4-7 & cleanup
  // Test 4: R-type SRA (arithmetic
       right shift)
  // Test 5: I-type ANDI (imm vs reg
       )
  // Test 6: LUI (is_lui=1; expect
       operand2)
  // Test 7: AUIPC (is_auipc=1;
       expect pc+operand2)

  $display("All␣ALU␣tests␣passed");
  #1; $finish;
end

endmodule
```

Figure: Testbench of ALU (cont'd)

ALU

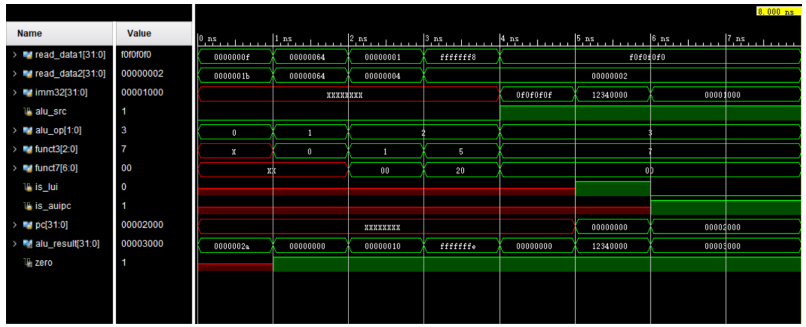

Figure: Waveform of ALU

ALU 波形验证

- Test 1 (`alu_op=2'b00`, `alu_src=0`): **加法** → `alu_result=0x0000002A`, `zero=0`
- Test 2 (`alu_op=2'b01`): **分支减法** → `zero=1`
- Test 3 (`alu_op=2'b10`, `funct3=3'b001`): **逻辑左移**
- Test 4 (`alu_op=2'b10`, `funct3=3'b101`, `funct7[5]=1`): **算术右移** → `alu_result=0xFFFFFFFE`

ALU 波形验证 (cont'd)

- Test 5 (`alu_op=2'b11`, `funct3=3'b111`): ANDI → `alu_result=0x00000000`
- Test 6 (`is_lui=1`): LUI → `alu_result=0x12340000`
- Test 7 (`is_auipc=1`): AUIPC → `alu_result=pc+imm=0x00003000`

经核查上述运算结果均正确。